In [3]:
```
import pandas as pd

# 1. Load the dataset
df = pd.read_csv("/Users/rederr/Desktop/Tp_Data/BigData/Projet/DataSet/Clear

# 2. Show first rows
df.head()
```

Out[3]:

	Unnamed: 0	airline	flight	source_city	departure_time	stops	arrival_time	dest
0	0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	
1	1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	
2	2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	
3	3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	
4	4	Vistara	UK-963	Delhi	Morning	zero	Morning	

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300153 entries, 0 to 300152
Data columns (total 12 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Unnamed: 0        300153 non-null  int64
 1   airline           300153 non-null  object
 2   flight            300153 non-null  object
 3   source_city       300153 non-null  object
 4   departure_time    300153 non-null  object
 5   stops             300153 non-null  object
 6   arrival_time      300153 non-null  object
 7   destination_city  300153 non-null  object
 8   class             300153 non-null  object
 9   duration          300153 non-null  float64
 10  days_left         300153 non-null  int64
 11  price             300153 non-null  int64
dtypes: float64(1), int64(3), object(8)
memory usage: 27.5+ MB
```

In [5]:
```
df.describe()
```

Out[5]:

	Unnamed: 0	duration	days_left	price
count	300153.000000	300153.000000	300153.000000	300153.000000
mean	150076.000000	12.221021	26.004751	20889.660523
std	86646.852011	7.191997	13.561004	22697.767366
min	0.000000	0.830000	1.000000	1105.000000
25%	75038.000000	6.830000	15.000000	4783.000000
50%	150076.000000	11.250000	26.000000	7425.000000
75%	225114.000000	16.170000	38.000000	42521.000000
max	300152.000000	49.830000	49.000000	123071.000000

In [6]:
```
# Categorical columns (object)
cat_cols = df.select_dtypes(include=["object"]).columns.tolist()

# Numerical columns (int/float)
num_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()

cat_cols, num_cols
```

Out[6]:
```
(['airline',
  'flight',
  'source_city',
  'departure_time',
  'stops',
  'arrival_time',
  'destination_city',
  'class'],
 ['Unnamed: 0', 'duration', 'days_left', 'price'])
```

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
Unnamed: 0          0
airline             0
flight              0
source_city         0
departure_time      0
stops               0
arrival_time        0
destination_city    0
class               0
duration            0
days_left           0
price               0
dtype: int64
```

In [8]:
```
for col in ['departure_time', 'arrival_time', 'stops']:
    print(col, df[col].unique())
```

```
departure_time ['Evening' 'Early_Morning' 'Morning' 'Afternoon' 'Night' 'Lat
e_Night']
arrival_time ['Night' 'Morning' 'Early_Morning' 'Afternoon' 'Evening' 'Late_
Night']
stops ['zero' 'one' 'two_or_more']
```

In [9]:
```
for col in ['airline', 'source_city', 'destination_city', 'class']:
    print(col, df[col].unique())
```

```
airline ['SpiceJet' 'AirAsia' 'Vistara' 'GO_FIRST' 'Indigo' 'Air_India']
source_city ['Delhi' 'Mumbai' 'Bangalore' 'Kolkata' 'Hyderabad' 'Chennai']
destination_city ['Mumbai' 'Bangalore' 'Kolkata' 'Hyderabad' 'Chennai' 'Delh
i']
class ['Economy' 'Business']
```

In [10]:
```
time_order_1 = {
    'Evening': 0,
    'Early_Morning': 1,
    'Morning': 2,
    'Afternoon': 3,
    'Night': 4,
    'Late_Night': 5
}
time_order_2 = {
    'Early_Morning': 0,
    'Morning': 1,
    'Afternoon': 2,
    'Evening': 3,
    'Night': 4,
    'Late_Night': 5
}

df['departure_time'] = df['departure_time'].map(time_order_1)
df['arrival_time'] = df['arrival_time'].map(time_order_2)
df.head()
```

Out[10]:

	Unnamed: 0	airline	flight	source_city	departure_time	stops	arrival_time	desti
0	0	SpiceJet	SG-8709	Delhi	0	zero	4	
1	1	SpiceJet	SG-8157	Delhi	1	zero	1	
2	2	AirAsia	I5-764	Delhi	1	zero	0	
3	3	Vistara	UK-995	Delhi	2	zero	2	
4	4	Vistara	UK-963	Delhi	2	zero	1	

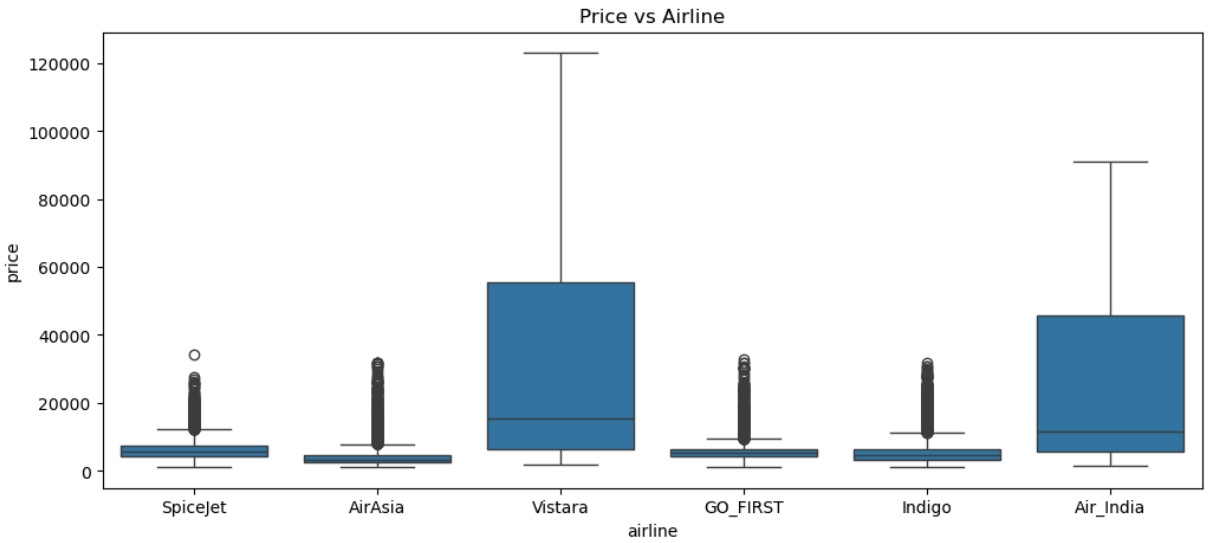
In [11]:
```
stops_map = {'zero': 0, 'one': 1, 'two_or_more': 2}
df['stops'] = df['stops'].map(stops_map)
```

In [12]:
```
df_encoded = pd.get_dummies(
    df,
    columns = ['airline', 'source_city', 'destination_city', 'class', 'fligh
    drop_first = True
)
```

In [13]:
```
df_encoded = df_encoded.drop(columns = ['Unnamed: 0'])
```

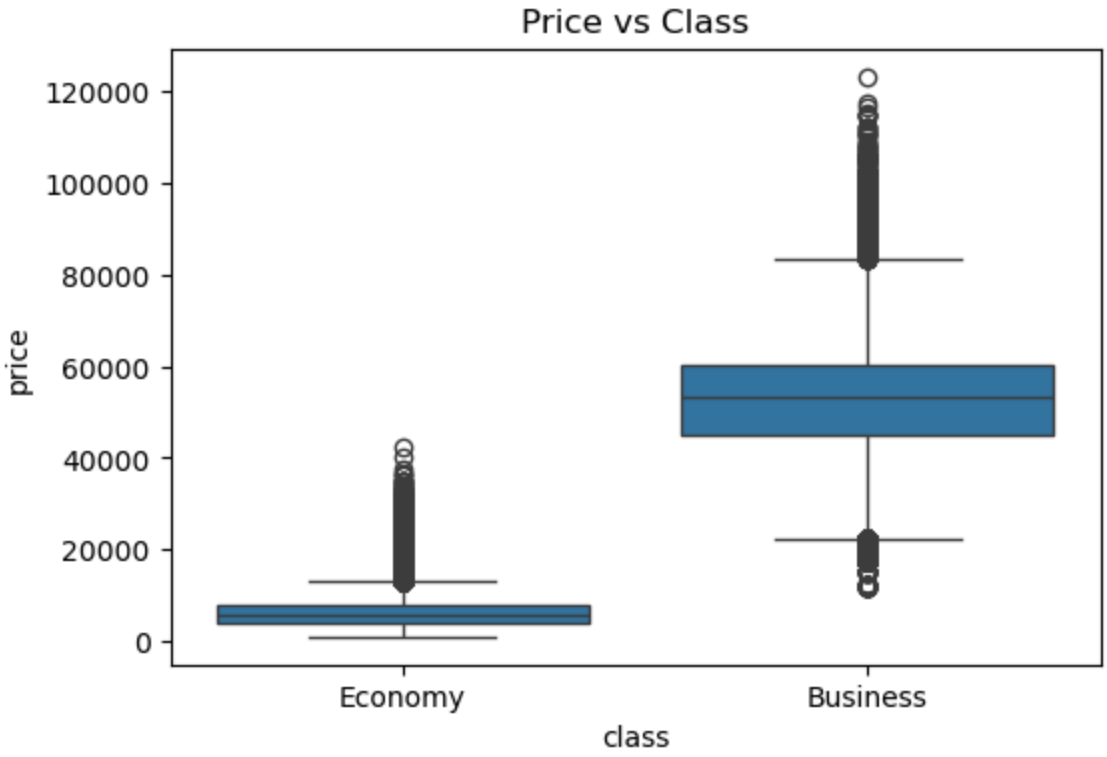
In [14]:
```
import matplotlib.pyplot as plt
import seaborn as sns

#Price vs Airline
plt.figure(figsize = (12, 5))
sns.boxplot(x = 'airline', y = 'price', data = df)
plt.title('Price vs Airline')
plt.show()
```

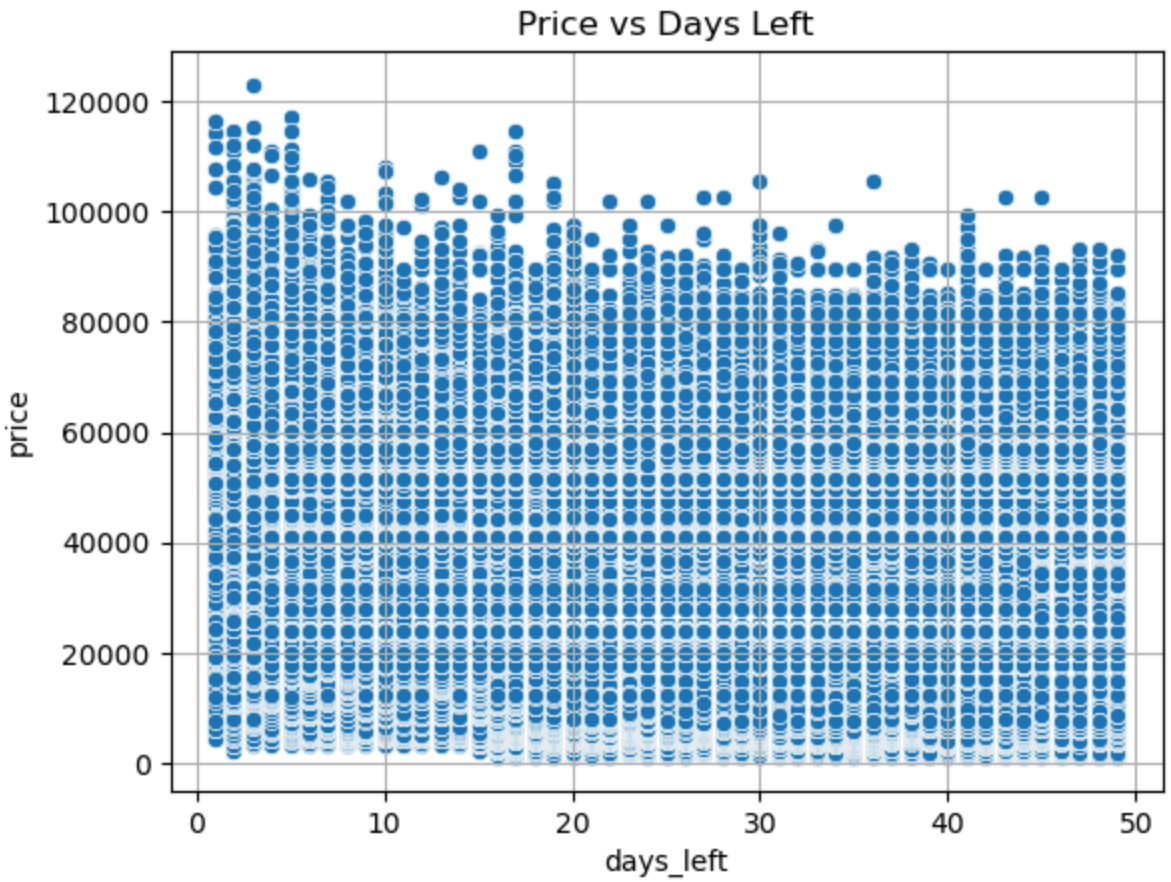


In [15]:
```
#Price vs Class
plt.figure(figsize=(6, 4))

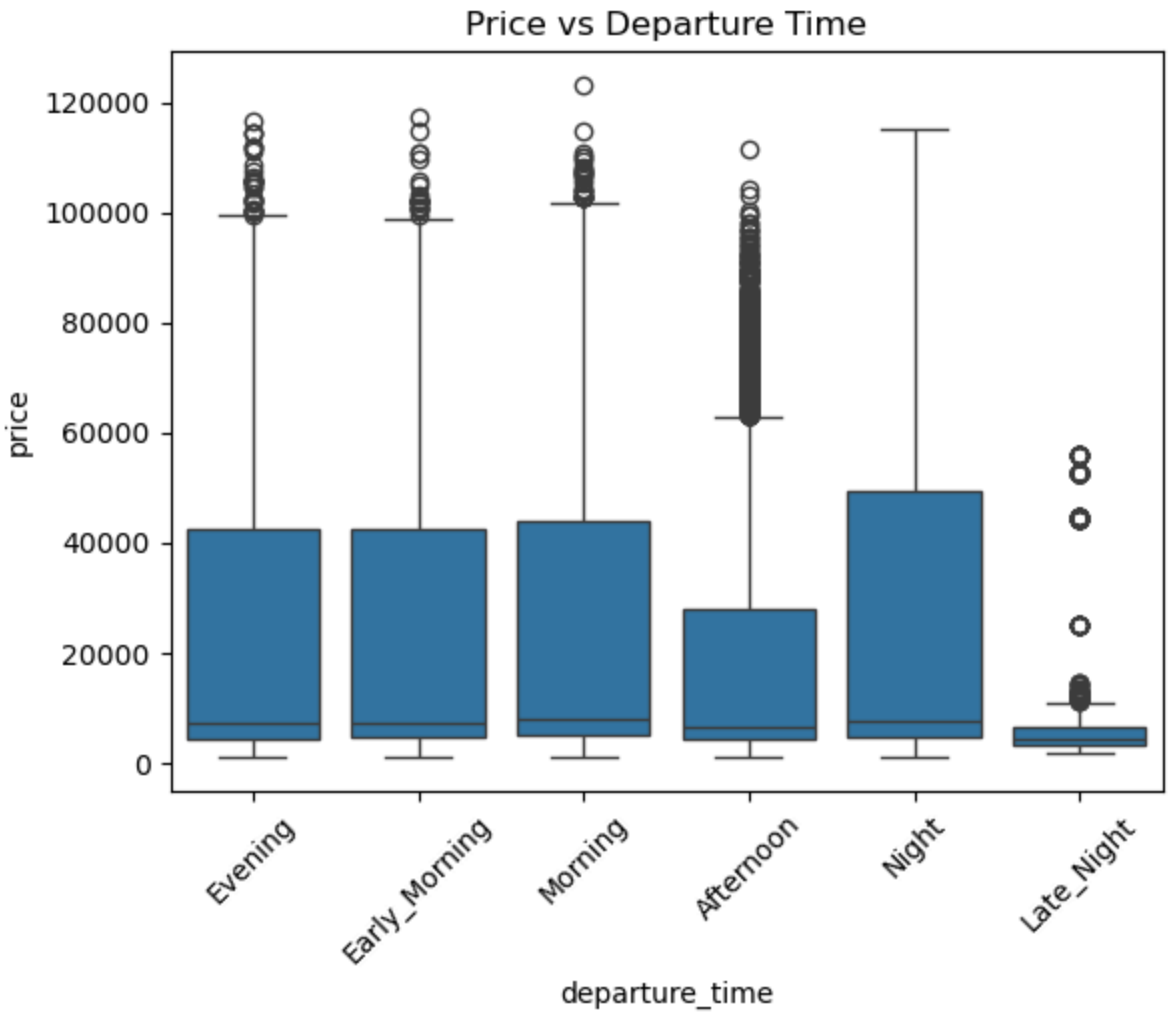
sns.boxplot(x = 'class', y = 'price', data = df)
plt.title('Price vs Class')
plt.show()
```



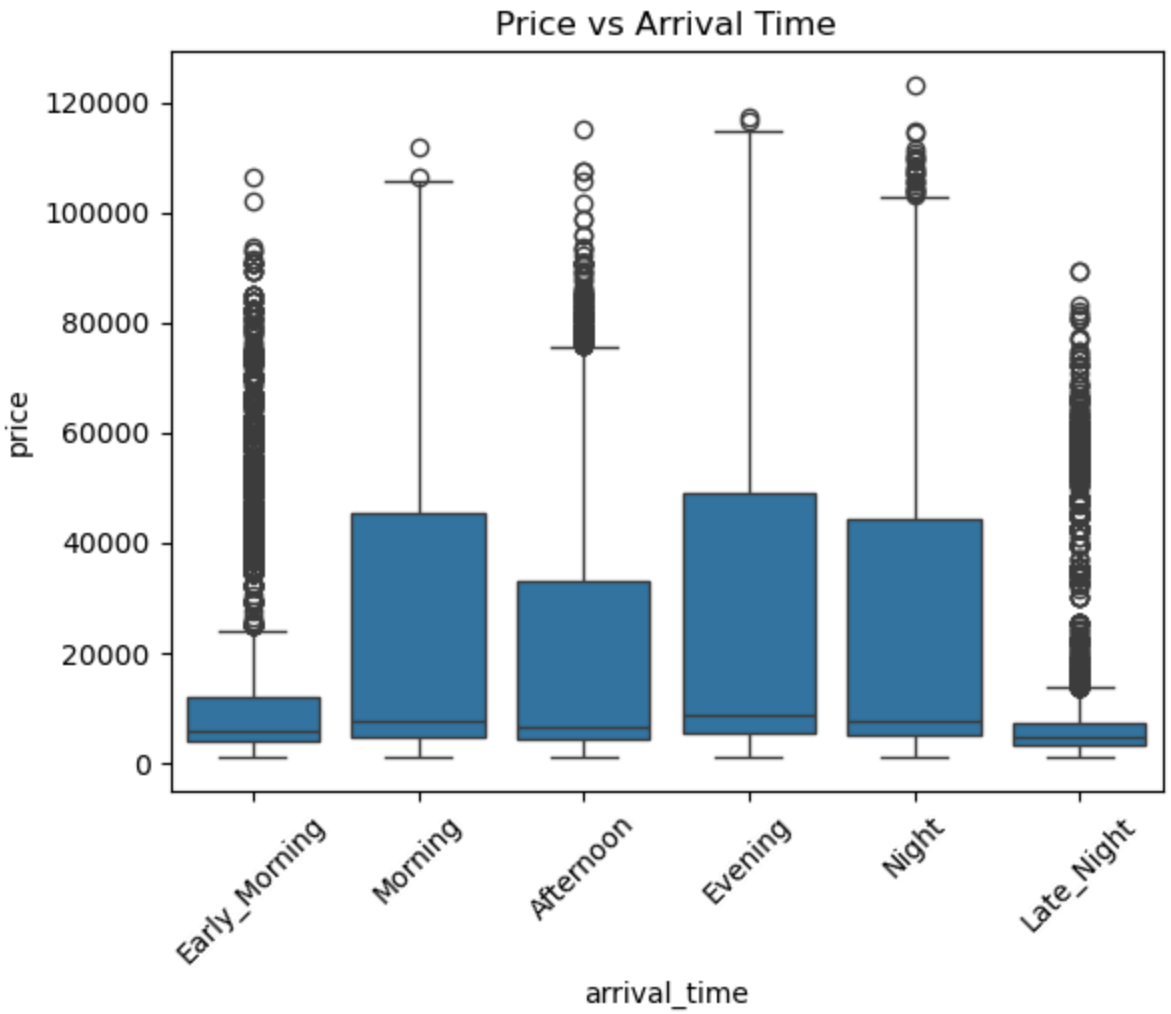
In [16]:
```
#Price vs Days Left
sns.scatterplot(x = df['days_left'], y = df['price'])
plt.title('Price vs Days Left')
plt.grid(True)
plt.show()
```



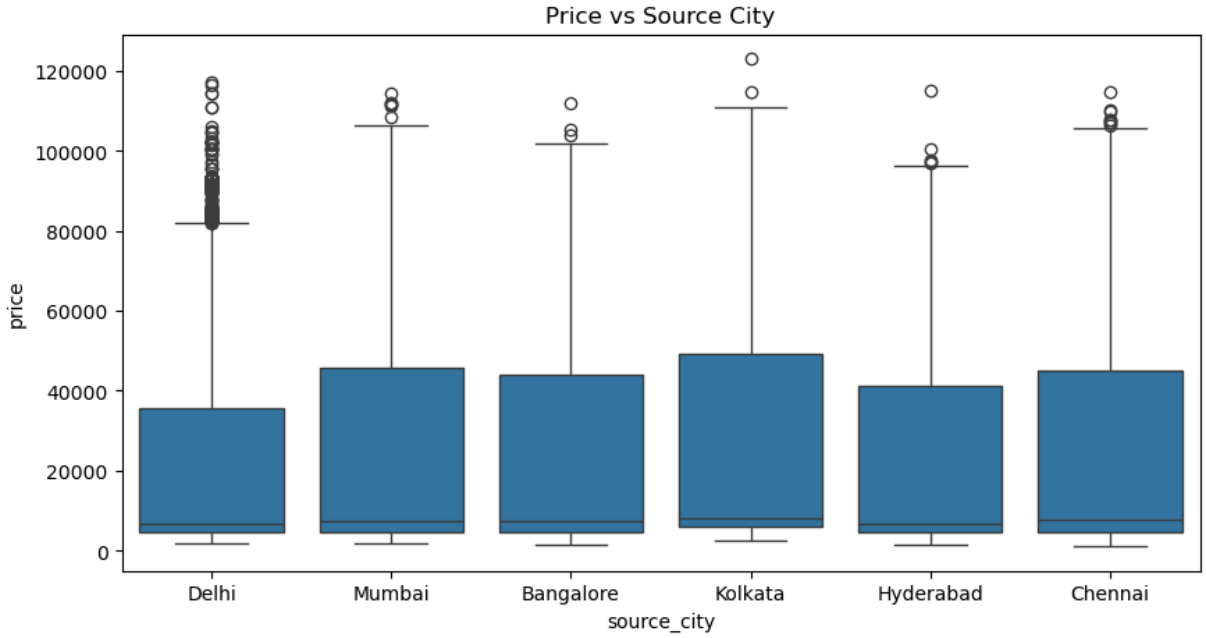
In [17]:
```
#Price vs Departure Time
labels = ['Evening', 'Early_Morning', 'Morning', 'Afternoon', 'Night', 'Late
sns.boxplot(x = 'departure_time', y = 'price', data = df)
plt.xticks(ticks = [0, 1, 2, 3, 4, 5], labels = labels, rotation = 45)
plt.title('Price vs Departure Time')
plt.show()
```



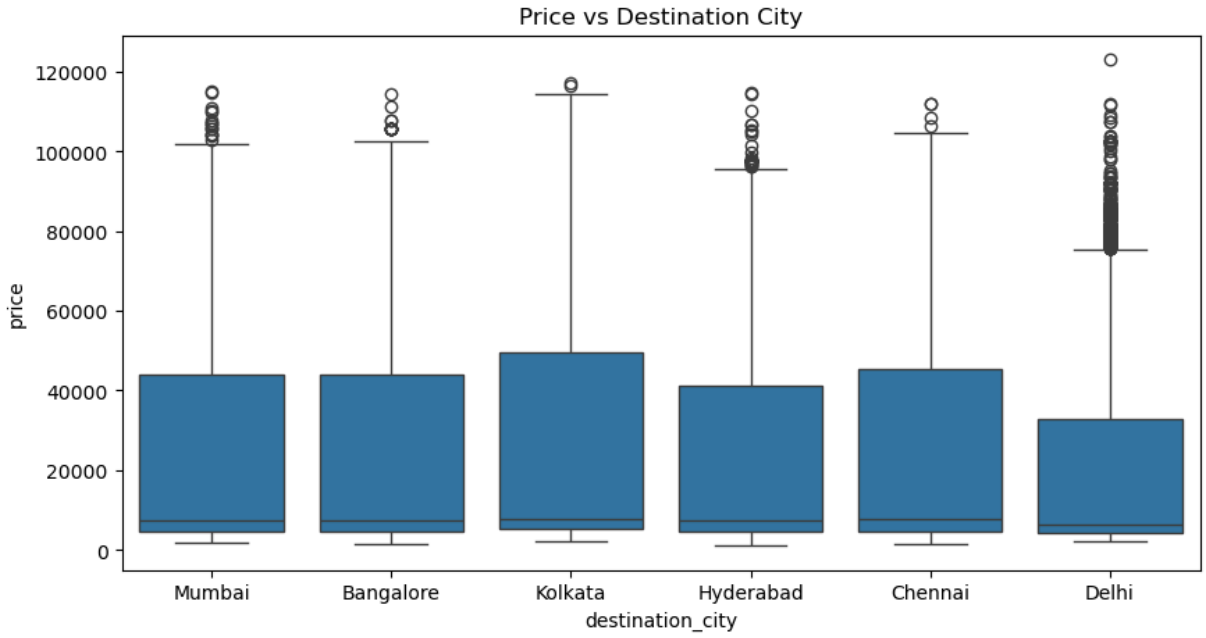
In [18]:
```
#Price vs Arrival Time
labels = ['Early_Morning', 'Morning', 'Afternoon', 'Evening', 'Night', 'Late
sns.boxplot(x = 'arrival_time', y = 'price', data = df)
plt.xticks(ticks = [0, 1, 2, 3, 4, 5], labels = labels, rotation = 45)
plt.title('Price vs Arrival Time')
plt.show()
```



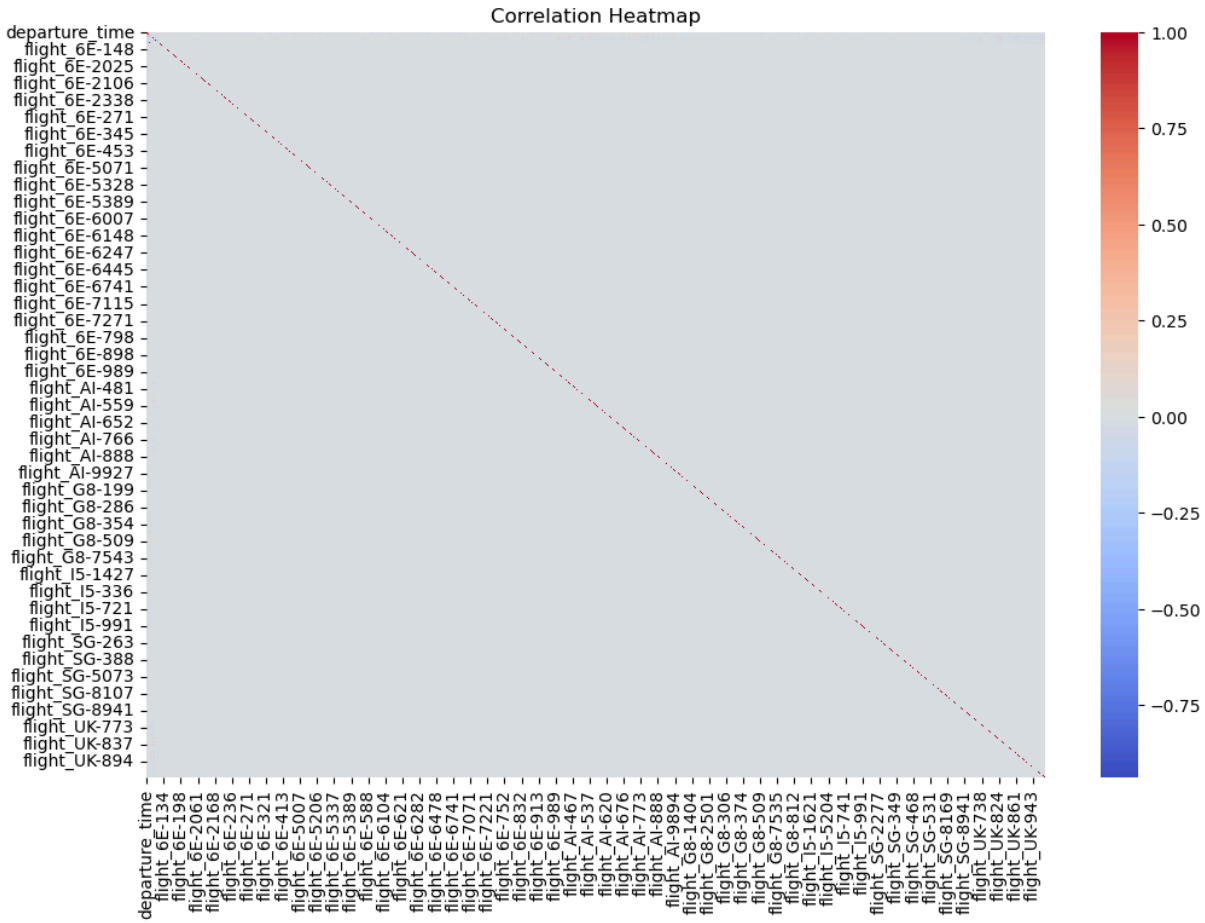
In [19]:
```
#Price vs Source City
plt.figure(figsize = (10, 5))
sns.boxplot(x = 'source_city', y = 'price', data = df)
plt.title('Price vs Source City')
plt.show()
```



In [20]:
```
#Price vs Destination City
plt.figure(figsize = (10, 5))
sns.boxplot(x = 'destination_city', y = 'price', data = df)
plt.title('Price vs Destination City')
plt.show()
```



In [21]:
```
plt.figure(figsize = (12, 8))
corr = df_encoded.corr()
sns.heatmap(corr, cmap = 'coolwarm', annot = False)
plt.title('Correlation Heatmap')
plt.show()
```



In [22]:
```
corr_with_price = corr['price'].sort_values(ascending = False)
corr_with_price
```

Out[22]:
```
price                1.000000
airline_Vistara      0.360816
duration             0.204222
stops                0.119648
airline_Air_India    0.070041
                         ...
days_left           -0.091949
airline_SpiceJet    -0.114019
airline_GO_FIRST    -0.194179
airline_Indigo      -0.280882
class_Economy       -0.937860
Name: price, Length: 1582, dtype: float64
```

In [23]:
```
# Threshold to keep features moderately correlated with price
threshold = 0.05

# Select all features that meet the condition
selected_corr_features = corr_with_price[abs(corr_with_price) >= threshold].

# Remove target variable
selected_corr_features.remove('price')

# Show selected features
selected_corr_features
```

Out[23]:
```
['airline_Vistara',
 'duration',
 'stops',
 'airline_Air_India',
 'flight_UK-772',
 'flight_UK-706',
 'flight_UK-810',
 'flight_UK-930',
 'flight_UK-940',
 'flight_UK-970',
 'flight_UK-808',
 'flight_UK-870',
 'flight_UK-820',
 'flight_UK-996',
 'flight_UK-988',
 'flight_UK-836',
 'destination_city_Delhi',
 'days_left',
 'airline_SpiceJet',
 'airline_GO_FIRST',
 'airline_Indigo',
 'class_Economy']
```

In [24]:
```
# SelectKBest (Statistical Feature Selection)
from sklearn.feature_selection import SelectKBest, f_regression
X = df_encoded.drop('price', axis = 1)
y = df_encoded['price']
selector = SelectKBest(score_func = f_regression, k = 10)
selector.fit(X, y)
kbest_features = X.columns[selector.get_support()].tolist()
kbest_features
```

Out[24]:
```
['stops',
 'duration',
 'days_left',
 'airline_Air_India',
 'airline_GO_FIRST',
 'airline_Indigo',
 'airline_SpiceJet',
 'airline_Vistara',
 'class_Economy',
 'flight_UK-772']
```

In [25]:
```
# RandomForest Feature Importance
from sklearn.ensemble import RandomForestRegressor

# Train a RandomForest model to evaluate feature importance
model = RandomForestRegressor(n_estimators = 200, random_state = 42)
model.fit(X, y)

# Store feature importances in a Series
importances = pd.Series(model.feature_importances_, index = X.columns)

# Display the top 10 most important features
```

```
top_rf_features = importances.sort_values(ascending = False).head(10)
top_rf_features
```

Out[25]:
```
class_Economy              0.879730
duration                   0.048408
days_left                  0.018042
airline_Vistara            0.005287
airline_Air_India          0.004994
destination_city_Delhi     0.003840
source_city_Delhi          0.003347
arrival_time               0.002724
source_city_Mumbai         0.002080
destination_city_Mumbai    0.001977
dtype: float64
```

In [26]:
```
# Combine features selected by all three methods
final_features = list(
    set(selected_corr_features
        + kbest_features
        + top_rf_features.index.tolist()
       )
)

# Show the final feature list for modeling
final_features
```

Out[26]:
```
['airline_Indigo',
 'flight_UK-970',
 'stops',
 'duration',
 'days_left',
 'airline_SpiceJet',
 'flight_UK-820',
 'airline_Air_India',
 'flight_UK-988',
 'flight_UK-870',
 'flight_UK-772',
 'flight_UK-706',
 'arrival_time',
 'airline_Vistara',
 'flight_UK-810',
 'airline_GO_FIRST',
 'destination_city_Delhi',
 'flight_UK-808',
 'destination_city_Mumbai',
 'source_city_Delhi',
 'flight_UK-930',
 'source_city_Mumbai',
 'flight_UK-940',
 'flight_UK-836',
 'class_Economy',
 'flight_UK-996']
```

In [27]:
```
# Remove all flight_xxx features from correlation-selected features
selected_corr_features = [f for f in selected_corr_features if not f.startsw
```

```
# Remove flight_xxx from SelectKBest
kbest_features = [f for f in kbest_features if not f.startswith('flight_')]

# Remove flight_xxx from RandomForest top features
top_rf_features_clean = [f for f in top_rf_features.index.tolist() if not f.

selected_corr_features, kbest_features, top_rf_features_clean
```

Out[27]:

```
(['airline_Vistara',
  'duration',
  'stops',
  'airline_Air_India',
  'destination_city_Delhi',
  'days_left',
  'airline_SpiceJet',
  'airline_GO_FIRST',
  'airline_Indigo',
  'class_Economy'],
 ['stops',
  'duration',
  'days_left',
  'airline_Air_India',
  'airline_GO_FIRST',
  'airline_Indigo',
  'airline_SpiceJet',
  'airline_Vistara',
  'class_Economy'],
 ['class_Economy',
  'duration',
  'days_left',
  'airline_Vistara',
  'airline_Air_India',
  'destination_city_Delhi',
  'source_city_Delhi',
  'arrival_time',
  'source_city_Mumbai',
  'destination_city_Mumbai'])
```

In [28]:

```
final_features_clean = list(
    set(
        selected_corr_features
        + kbest_features
        + top_rf_features_clean
    )
)

final_features_clean = sorted(final_features_clean)
final_features_clean
```

Out[28]:
```
['airline_Air_India',
 'airline_GO_FIRST',
 'airline_Indigo',
 'airline_SpiceJet',
 'airline_Vistara',
 'arrival_time',
 'class_Economy',
 'days_left',
 'destination_city_Delhi',
 'destination_city_Mumbai',
 'duration',
 'source_city_Delhi',
 'source_city_Mumbai',
 'stops']
```

In [29]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressc
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

import xgboost as xgb
from sklearn.metrics import mean_squared_error, r2_score
```

In [30]:
```
X = df_encoded[final_features_clean]
y = df_encoded['price']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, r
```

In [31]:
```
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state = 42),
    'Random Forest': RandomForestRegressor(n_estimators = 200, random_state
    'Gradient Boosting': GradientBoostingRegressor(random_state = 42),
    'KNN Regressor': KNeighborsRegressor(n_neighbors = 5),
    'Support Vector Regressor': SVR(),
    'Extra Trees Regressor': ExtraTreesRegressor(n_estimators = 200, random_
        }
```

In [32]:
```
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)

    results.append([name, rmse, r2])
```

In [33]:
```
results_df = pd.DataFrame(results, columns = ['Model', 'RMSE', 'R2 Score'])
```

```
results_df.sort_values(by = 'R2 Score', ascending = False).head(7)
```

Out[33]:

	Model	RMSE	R2 Score
2	Random Forest	3753.753721	0.972665
6	Extra Trees Regressor	4196.659210	0.965834
1	Decision Tree	4517.816547	0.960405
3	Gradient Boosting	5178.166718	0.947984
0	Linear Regression	6916.556210	0.907196
4	KNN Regressor	7009.154345	0.904695
5	Support Vector Regressor	25838.593835	-0.295162

In [35]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
from sklearn.model_selection import cross_val_score

evaluation_results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    mae = mean_absolute_error(y_test, preds)
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)
    cv = cross_val_score(model, X_train, y_train, cv = 5, scoring = 'r2').me

    evaluation_results.append([name, mae, mse, rmse, r2, cv])

eval_df = pd.DataFrame(
    evaluation_results,
    columns = ['Model', 'MAE', 'MSE', 'RMSE', 'R2', 'CV Score']
)

eval_df.sort_values(by = 'R2', ascending = False)
```

Out[35]:

	Model	MAE	MSE	RMSE	R2	CV Score
2	Random Forest	1701.612190	1.409067e+07	3753.753721	0.972665	0.974311
6	Extra Trees Regressor	1985.172776	1.761195e+07	4196.659210	0.965834	0.967318
1	Decision Tree	1888.307915	2.041067e+07	4517.816547	0.960405	0.962071
3	Gradient Boosting	3050.885511	2.681341e+07	5178.166718	0.947984	0.948896
0	Linear Regression	4523.310723	4.783875e+07	6916.556210	0.907196	0.907395
4	KNN Regressor	4343.921167	4.912824e+07	7009.154345	0.904695	0.896180
5	Support Vector Regressor	15367.316065	6.676329e+08	25838.593835	-0.295162	-0.300079

In [37]:
```
import pickle

# Train the best model again using full training data
best_model = RandomForestRegressor(n_estimators = 200, random_state = 42, n_
best_model.fit(X_train, y_train)

# Save as a .pkl file
with open('flight_price_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

print('Model saved as flight_price_model.pkl')
```

```
Model saved as flight_price_model.pkl
```

In [ ]: